

Sorting Templates

All recursive sorting uses **half-open intervals** `[start, end)` throughout.

Inside merge and partition: `i`, `j`, `k` as scan/write pointers (consistent with binary search convention).

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
912	Sort an Array	Sort an integer array. No built-in sort allowed.	Divide and conquer — either split-then-merge (merge sort) or partition-then-recurse (quick sort); both reach $O(n \log n)$.	$O(n \log n)$	$O(n)$ merge / $O(\log n)$ quick	Standard
215	Kth Largest Element in an Array	Return the k-th largest element without sorting the full array.	Quick select partitions around a pivot and only recurses into the side containing the target rank, so it averages $O(n)$ instead of fully sorting.	$O(n)$ avg	$O(\log n)$	Standard
973	K Closest Points to Origin	Return the k closest points to origin (0,0). Any order.	Same quick select, but partition on squared distance — once the first k slots are filled with the closest points, their internal order doesn't matter.	$O(n)$ avg	$O(\log n)$	Standard
148	Sort List	Sort a linked list in $O(n \log n)$ time, $O(\log n)$ space.	Merge sort fits linked lists — there's no random access for quick sort, but splitting at the middle and merging two sorted lists needs only pointer rewiring.	$O(n \log n)$	$O(\log n)$	Standard
179	Largest Number	Given a list of non-negative integers, arrange them to form the largest number.	Order two numbers by which concatenation is bigger ($b+a$ vs $a+b$) — this pairwise rule sorts the whole list into the largest possible string.	$O(n \log n)$	$O(n)$	Standard
315	Count of Smaller Numbers After Self	For each element, count how many elements to its right are smaller.	During a merge, whenever a left-half element is placed, every right-half element already emitted is both smaller and to its right — so the merge counts the smaller-after-self pairs for free.	$O(n \log n)$	$O(n)$	Standard
56	Merge Intervals	Merge all overlapping intervals. Return array of non-overlapping intervals.	After sorting by start, overlaps can only be with the most recent interval, so one scan either extends its end or appends a new one.	$O(n \log n)$	$O(n)$	Standard
88	Merge Sorted Array	Given two sorted arrays <code>nums1</code> (with capacity for $m+n$) and <code>nums2</code> , merge <code>nums2</code> into <code>nums1</code> in-place.		$O(m+n)$	$O(1)$	Fill from the END backwards using three pointers <code>i</code> (end of <code>nums1</code> valid data), <code>j</code> (end of <code>nums2</code>), <code>k</code> (end of merged result). This avoids the need to shift elements.

#	Name	Description	Intuition	Time	Space	Key Implementation
327	Count of Range Sum	Given an integer array, count the number of range sums <code>sum(i, j)</code> that lie within <code>[lower, upper]</code> (inclusive).		$O(n \log n)$	$O(n)$	Build prefix sum array. Then use modified merge sort — during the merge phase, for each element in the left half, use two sliding pointers <code>j</code> and <code>k</code> into the right half to count valid prefix sums (those where <code>arr[k] - arr[i]</code> falls in range). This gives $O(n \log n)$ vs $O(n^2)$ brute force.
164	Maximum Gap	Given an unsorted array, return the maximum difference between successive elements in its sorted form. Must run in linear time.		$O(n)$	$O(n)$	bucket sort by pigeonhole principle. With n elements, the max gap is at least <code>ceil((max-min)/(n-1))</code> . Place elements into buckets of that size; the max gap spans between one bucket's max and the next non-empty bucket's min.

Template 1 — Merge Sort

Variables: `[start, end)` half-open range being sorted · `mid` = split point · `temp` = scratch buffer · `i` = left scan pointer `[start, mid)` · `j` = right scan pointer `[mid, end)` · `k` = write pointer into temp **Pseudocode:**

```
mergeSort(nums, start, end):
    if range has 0 or 1 elements: return          // already sorted
    mid = midpoint of [start, end)
    mergeSort left half [start, mid)
    mergeSort right half [mid, end)
    merge the two sorted halves

merge:
    i = start, j = mid, k = start
    while either half has elements left:
        pick smaller front of the two halves (left wins ties → stable)
        write it to temp[k], advance that half's pointer and k
    copy temp back into nums[start, end)
```

```
// split in half until trivially sorted, then merge two sorted halves into one. – WHEN: need stable so
// Entry point
public void mergeSort(int[] nums) {
    mergeSort(nums, 0, nums.length, new int[nums.length]);
}

// [start, end) half-open
private void mergeSort(int[] nums, int start, int end, int[] temp) {
    if (end - start <= 1) return; // base case: end - start <= 1 → 0 or 1 elements, already sort
    int mid = start + (end - start) / 2;
    mergeSort(nums, start, mid, temp); // sort [start, mid)
    mergeSort(nums, mid, end, temp); // sort [mid, end)
    merge(nums, start, mid, end, temp);
}

// i scans left [start,mid), j scans right [mid,end), k writes to temp
private void merge(int[] nums, int start, int mid, int end, int[] temp) {
    int i = start, j = mid, k = start;
    while (i < mid || j < end) {
        if (j >= end || (i < mid && nums[i] <= nums[j])) {
            temp[k++] = nums[i++];
        } else {
            temp[k++] = nums[j++];
        }
    }
    for (i = start; i < end; i++) nums[i] = temp[i];
}
```

Template 2 — Quick Sort (3-way Dutch Flag Partition)

Variables: [start, end) half-open range · pivot = chosen partition value · i = boundary of <pivot region · k = scan/cursor pointer · j = boundary of >pivot region (from the right) **Pseudocode:**

```
quickSort(nums, start, end):
    if range has 0 or 1 elements: return
    pivot = middle element's value
    i = start, k = start, j = end-1 // [start,i)<p | [i,k)==p | [k,j] unknown | (j,end)>p
    while k <= j:
        if nums[k] < pivot: swap into < region, advance k and i
        elif nums[k] == pivot: advance k
        else: swap to > region, shrink j (don't advance k)
    recurse on < region [start, i)
    recurse on > region [k, end) // ==pivot middle is final
```

```

// pick a pivot, partition into <pivot | ==pivot | >pivot, recurse on the two ends only. - WHEN: in-pl
// Entry point - no extra array needed (in-place)
public void quickSort(int[] nums) {
    quickSort(nums, 0, nums.length);
}

// [start, end) half-open
private void quickSort(int[] nums, int start, int end) {
    if (end - start <= 1) return;
    int pivot = nums[start + (end - start) / 2];
    int i = start, k = start, j = end - 1;
    // Invariant: [start,i) < pivot | [i,k) == pivot | [k,j) unknown | (j,end) > pivot
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k++, i++);
        } else if (nums[k] == pivot) {
            k++;
        } else {
            swap(nums, k, j--);
        }
    }
    // After: [start,i) < pivot, [i,k) == pivot, [k,end) > pivot
    quickSort(nums, start, i); // recurse on < region
    quickSort(nums, k, end); // recurse on > region
}

private void swap(int[] nums, int i, int j) {
    int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
}

```

Template 3 — Quick Select (partial sort — k-th element)

Extends quick sort: after partition, only recurse into the side that contains `target` index.

Variables: `[start, end)` half-open range · `target` = 0-indexed position we want settled · `pivot` = partition value · `i` = boundary of `<pivot` region · `k` = scan/cursor pointer · `j` = boundary of `>pivot` region **Pseudocode:**

```

quickSelect(nums, start, end, target):
    pivot = middle element's value
    partition into [start,i)<p | [i,k)==p | [k,end)>p // same 3-way as quick sort
    if target < i:        recurse into left [start, i)
    elif target < k:      target sits in ==pivot region → return pivot
    else:                recurse into right [k, end)

```

```
// same partition as quick sort, but recurse into only the one side holding target → O(n) avg. – WHEN:
// Returns value at 0-indexed position target after sorting
private int quickSelect(int[] nums, int start, int end, int target) {
    int pivot = nums[start + (end - start) / 2];
    int i = start, k = start, j = end - 1;
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k++, i++);
        } else if (nums[k] == pivot) {
            k++;
        } else {
            swap(nums, k, j--);
        }
    }
    // [start,i) < pivot, [i,k) == pivot, [k,end) > pivot
    if (target < i) {
        return quickSelect(nums, start, i, target);
    } else if (target < k) {
        return pivot; // target lands in == region
    } else {
        return quickSelect(nums, k, end, target);
    }
}
```

Template 4 — Counting Sort

$O(n + \text{range})$. Use when values are bounded integers.

Variables: `min / max` = value range bounds · `buckets[v-min]` = count of value `v` · `i` = write pointer back into `arr` · `v` = current value index into buckets **Pseudocode:**

```
if array empty: return
find min and max values
buckets = array sized (max-min+1), all zero
for each x in arr: increment buckets[x-min] // tally occurrences
i = 0
for v from 0 to last bucket:
    while bucket v still has count: write (v+min) into arr[i], advance i
```

```
// tally how many times each value occurs, then emit values in order – no comparisons. – WHEN: integer
public void countingSort(int[] arr) {
    if (arr.length == 0) return;
    int min = Arrays.stream(arr).min().getAsInt();
    int max = Arrays.stream(arr).max().getAsInt();
    int[] buckets = new int[max - min + 1];
    for (int x : arr) buckets[x - min]++;
    int i = 0;
    for (int v = 0; v < buckets.length; v++)
        while (buckets[v]-- > 0) arr[i++] = v + min;
}
```

#912 Sort an Array

Description: Sort an integer array. No built-in sort allowed.

Intuition: Divide and conquer — either split-then-merge (merge sort) or partition-then-recurse (quick sort); both reach $O(n \log n)$

log n).

Variables: [start, end) half-open range · mid = split point · temp = merge scratch buffer · pivot = quick-sort partition value · i / j / k = scan and write pointers **Pseudocode:**

Merge Sort version:

mergeSort [0, n): split at mid, sort both halves, merge them

merge: walk i over left, j over right, write smaller into temp[k], copy temp back

Quick Sort version:

quickSort [0, n): pick middle pivot, 3-way partition <p | ==p | >p

recurse on the < region [start,i) and the > region [k,end)

```
// Merge Sort version
class Solution {
    public int[] sortArray(int[] nums) {
        mergeSort(nums, 0, nums.length, new int[nums.length]);
        return nums;
    }
    private void mergeSort(int[] nums, int start, int end, int[] temp) {
        if (end - start <= 1) return;
        int mid = start + (end - start) / 2;
        mergeSort(nums, start, mid, temp);
        mergeSort(nums, mid, end, temp);
        merge(nums, start, mid, end, temp);
    }
    private void merge(int[] nums, int start, int mid, int end, int[] temp) {
        int i = start, j = mid, k = start;
        while (i < mid || j < end) {
            if (j >= end || (i < mid && nums[i] <= nums[j])) {
                temp[k++] = nums[i++];
            } else {
                temp[k++] = nums[j++];
            }
        }
        for (i = start; i < end; i++) nums[i] = temp[i];
    }
}

// Quick Sort version (in-place, O(log n) stack)
class Solution {
    public int[] sortArray(int[] nums) {
        quickSort(nums, 0, nums.length);
        return nums;
    }
    private void quickSort(int[] nums, int start, int end) {
        if (end - start <= 1) return;
        int pivot = nums[start + (end - start) / 2];
        int i = start, k = start, j = end - 1;
        while (k <= j) {
            if (nums[k] < pivot) {
                swap(nums, k++, i++);
            } else if (nums[k] == pivot) {
                k++;
            } else {
                swap(nums, k, j--);
            }
        }
        quickSort(nums, start, i);
        quickSort(nums, k, end);
    }
    private void swap(int[] nums, int i, int j) {
        int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
    }
}
}
```

Time $O(n \log n)$ | **Space** $O(n)$ merge / $O(\log n)$ quick

#215 Kth Largest Element in an Array

Description: Return the k-th largest element without sorting the full array.

Intuition: Quick select partitions around a pivot and only recurses into the side containing the target rank, so it averages

$O(n)$ instead of fully sorting.

Key: k -th largest = $(n - k)$ -th smallest (0-indexed). Quick select, recurse one side only.

Variables: `target` = $n - k$ (0-indexed position of k -th largest) · `[start, end)` range · `pivot` = partition value · `i / k` = `< / ==` region boundaries · `j` = `>` boundary **Pseudocode:**

```
target = n - k // k-th largest is (n-k)-th smallest
quickSelect [0, n) for target:
    pick middle pivot, 3-way partition <p | ==p | >p
    if target < i: recurse left [start, i)
    elif target < k: return pivot // target lands in ==pivot region
    else:         recurse right [k, end)
```

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        return quickSelect(nums, 0, nums.length, nums.length - k);
    }
    private int quickSelect(int[] nums, int start, int end, int target) {
        int pivot = nums[start + (end - start) / 2];
        int i = start, k = start, j = end - 1;
        while (k <= j) {
            if (nums[k] < pivot) {
                swap(nums, k++, i++);
            } else if (nums[k] == pivot) {
                k++;
            } else {
                swap(nums, k, j--);
            }
        }
        if (target < i) {
            return quickSelect(nums, start, i, target);
        } else if (target < k) {
            return pivot;
        } else {
            return quickSelect(nums, k, end, target);
        }
    }
    private void swap(int[] nums, int i, int j) {
        int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
    }
}
```

Time $O(n)$ avg | **Space** $O(\log n)$

#973 K Closest Points to Origin

Description: Return the k closest points to origin $(0,0)$. Any order.

Intuition: Same quick select, but partition on squared distance — once the first k slots are filled with the closest points, their internal order doesn't matter.

Key: same quick select template; comparator = squared distance (avoid sqrt). After select, first k entries in `[0, k)` are the answer.

Variables: `k` = how many closest points wanted · `[start, end)` range · `pivot` = squared distance of pivot point · `i / current` = `< / ==` region boundaries · `j` = `>` boundary · `less / equal` = sizes of `< / ==` regions **Pseudocode:**

```

quickSelect partitions points so the closest k land in [0, k):
    if range size <= k: already done, return
    pivot = squared distance of middle point
    3-way partition by squared distance: <p | ==p | >p
    less = size of < region, equal = size of == region
    if k <= less:          recurse into < region
    elif k <= less+equal: first k already in place, return
    else:                 recurse into > region for remaining (k-less-equal)
answer = first k points

```

```

class Solution {
    public int[][] kClosest(int[][] pts, int k) {
        quickSelect(pts, 0, pts.length, k);
        return Arrays.copyOfRange(pts, 0, k);
    }
    private void quickSelect(int[][] pts, int start, int end, int k) {
        if (end - start <= k) return;
        long pivot = dist(pts[start + (end - start) / 2]);
        int i = start, current = start, j = end - 1;
        while (current <= j) {
            long d = dist(pts[current]);
            if (d < pivot) {
                swap(pts, current++, i++);
            } else if (d == pivot) {
                current++;
            } else {
                swap(pts, current, j--);
            }
        }
        // [start,i) < pivot, [i,current) == pivot, [current,end) > pivot
        int less = i - start, equal = current - i;
        if (k <= less) {
            quickSelect(pts, start, i, k);
        } else if (k <= less + equal) {
            return; // enough elements in [start,current)
        } else {
            quickSelect(pts, current, end, k - less - equal);
        }
    }
    private long dist(int[] p) { return (long) p[0]*p[0] + (long) p[1]*p[1]; }
    private void swap(int[][] pts, int i, int j) {
        int[] t = pts[i]; pts[i] = pts[j]; pts[j] = t;
    }
}

```

Time $O(n)$ avg | **Space** $O(\log n)$

#148 Sort List

Description: Sort a linked list in $O(n \log n)$ time, $O(\log n)$ space.

Intuition: Merge sort fits linked lists — there's no random access for quick sort, but splitting at the middle and merging two sorted lists needs only pointer rewiring.

Key: same merge sort structure — slow/fast pointers to find mid, split, sort halves, merge.

Variables: head = list start · slow / fast = pointers to locate the middle (fast moves 2x) · mid = head of second half · left / right = sorted halves · dummy / current = builder for the merged list **Pseudocode:**

```

sortList(head):
    if 0 or 1 nodes: return head
    advance slow by 1 and fast by 2 to find middle
    mid = slow.next; cut the list at slow (slow.next = null)
    left = sortList(first half), right = sortList(second half)
    return merge(left, right)
merge(a, b):
    walk both lists, append smaller node to current, advance
    attach whatever remains; return dummy.next

```

```

class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) return head;
        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next; fast = fast.next.next;
        }
        ListNode mid = slow.next;
        slow.next = null; // split into [head, slow] and [mid, ...]
        ListNode left = sortList(head);
        ListNode right = sortList(mid);
        return merge(left, right);
    }
    private ListNode merge(ListNode a, ListNode b) {
        ListNode dummy = new ListNode(0), current = dummy;
        while (a != null && b != null) {
            if (a.val <= b.val) {
                current.next = a;
                a = a.next;
            } else {
                current.next = b;
                b = b.next;
            }
            current = current.next;
        }
        current.next = (a != null) ? a : b;
        return dummy.next;
    }
}

```

Time $O(n \log n)$ | **Space** $O(\log n)$

#179 Largest Number

Description: Given a list of non-negative integers, arrange them to form the largest number.

Intuition: Order two numbers by which concatenation is bigger ($b+a$ vs $a+b$) — this pairwise rule sorts the whole list into the largest possible string.

Key: custom comparator — between a and b , prefer whichever concatenation $a+b$ vs $b+a$ is lexicographically larger.

Variables: $strs$ = numbers as strings · a / b = two strings being compared · comparator key = compare $b+a$ vs $a+b$

Pseudocode:

```

convert every number to a string
sort strings by: a before b if (b+a) < (a+b) // bigger concatenation comes first
if the largest string is "0": return "0" // all zeros case
join all strings together and return

```

```

class Solution {
    public String largestNumber(int[] nums) {
        String[] strs = new String[nums.length];
        for (int i = 0; i < nums.length; i++) strs[i] = String.valueOf(nums[i]);
        Arrays.sort(strs, (a, b) -> (b + a).compareTo(a + b));
        if (strs[0].equals("0")) return "0";    // all zeros
        return String.join("", strs);
    }
}

```

Time $O(n \log n)$ | **Space** $O(n)$

#315 Count of Smaller Numbers After Self

Description: For each element, count how many elements to its right are smaller.

Intuition: During a merge, whenever a left-half element is placed, every right-half element already emitted is both smaller and to its right — so the merge counts the smaller-after-self pairs for free.

Key: same merge sort template but track original indices. When element from left side is placed, $j - \text{mid}$ elements from the right side that have already been placed are all smaller and to its right.

Variables: `result[orig]` = smaller-after-self count per original index · `indices` = positions sorted by value (sort indices, not values) · `[start, end)` range · `mid` = split · `i / j` = left/right scan over indices · `k` = write pointer into temp **Pseudocode:**

```

indices = [0,1,...,n-1]
mergeSort over indices, comparing by nums[indices[..]]:
    merge: when placing a left element (nums[indices[i]] <= nums[indices[j]]):
        result[indices[i]] += (j - mid)    // right elements already emitted are smaller & to the right
    else place the right element
    copy merged indices back
collect result into a list

```

```

class Solution {
    int[] result, indices;

    public List<Integer> countSmaller(int[] nums) {
        int n = nums.length;
        result = new int[n];
        indices = new int[n];
        for (int i = 0; i < n; i++) indices[i] = i;
        mergeSort(nums, 0, n, new int[n]);
        List<Integer> output = new ArrayList<>();
        for (int r : result) output.add(r);
        return output;
    }

    private void mergeSort(int[] nums, int start, int end, int[] temp) {
        if (end - start <= 1) return;
        int mid = start + (end - start) / 2;
        mergeSort(nums, start, mid, temp);
        mergeSort(nums, mid, end, temp);
        merge(nums, start, mid, end, temp);
    }

    private void merge(int[] nums, int start, int mid, int end, int[] temp) {
        int i = start, j = mid, k = start;
        while (i < mid || j < end) {
            if (j >= end || (i < mid && nums[indices[i]] <= nums[indices[j]])) {
                result[indices[i]] += j - mid; // j-mid right-side elements already placed
                temp[k++] = indices[i++];
            } else {
                temp[k++] = indices[j++];
            }
        }
        for (i = start; i < end; i++) indices[i] = temp[i];
    }
}

```

Time $O(n \log n)$ | **Space** $O(n)$

#56 Merge Intervals

Description: Merge all overlapping intervals. Return array of non-overlapping intervals.

Intuition: After sorting by start, overlaps can only be with the most recent interval, so one scan either extends its end or appends a new one.

Key: sort by start time; greedily extend the last interval's end when overlap found.

Variables: intervals sorted by start · result = list of merged intervals · iv = current interval · last interval's [1] = end being extended **Pseudocode:**

```

sort intervals by start value
for each interval iv:
    if result empty OR last result's end < iv.start: // no overlap
        append iv as a new interval
    else: // overlap
        extend last result's end to max(last end, iv.end)
return result as array

```

```

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int[]> result = new ArrayList<>();
        for (int[] iv : intervals) {
            if (result.isEmpty() || result.get(result.size() - 1)[1] < iv[0]) {
                result.add(iv);
            } else {
                result.get(result.size() - 1)[1] = Math.max(result.get(result.size() - 1)[1], iv[1]);
            }
        }
        return result.toArray(new int[0][]);
    }
}

```

Time O(n log n) | **Space** O(n)

Algorithm Chooser

Signal	Algorithm
"sort array", stable, O(n) space OK	Merge Sort
"sort array", in-place, O(log n) space	Quick Sort
"k-th largest/smallest"	Quick Select — O(n) avg
"k closest / k most frequent"	Quick Select with custom comparator
"sort linked list"	Merge Sort (quick sort needs random access)
"count inversions / smaller after self"	Merge Sort — count during merge step
"arrange to maximize/compare"	Custom Sort comparator
"merge overlapping intervals"	Sort by start + linear scan
integers in bounded range	Counting Sort — O(n + range)

Half-open Interval Cheat Sheet

```

mergeSort(nums, start, end)  →  sorts [start, end)
    mid = start + (end-start)/2
    left:  [start, mid)
    right: [mid, end)
    base:  end - start <= 1

quickSort(nums, start, end)  →  sorts [start, end)
    after partition: [start,i) < pivot, [i,k) == pivot, [k,end) > pivot
    recurse: (start, i) and (k, end)

```

#88 Merge Sorted Array

Description: Given two sorted arrays `nums1` (with capacity for m+n) and `nums2` , merge `nums2` into `nums1` in-place.

Variation: Fill from the END backwards using three pointers `i` (end of `nums1` valid data), `j` (end of `nums2`), `k` (end of merged result). This avoids the need to shift elements.

Variables: `i` = last valid index of `nums1` (`m-1`) · `j` = last index of `nums2` (`n-1`) · `k` = write position at the very end of `nums1` (`m+n-1`) **Pseudocode:**

```
i = m-1, j = n-1, k = m+n-1          // fill from the back
while both halves have elements:
    write the larger of nums1[i], nums2[j] into nums1[k], move that pointer and k back
copy any remaining nums2 elements into the front of nums1
// remaining nums1 elements are already in place
```

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;    // ← VARIATION: three pointers from END
        while (i >= 0 && j >= 0)
            nums1[k--] = (nums1[i] >= nums2[j]) ? nums1[i--] : nums2[j--];
        while (j >= 0) nums1[k--] = nums2[j--];      // ← copy remaining nums2
    }
}
```

Time $O(m+n)$ | **Space** $O(1)$

#327 Count of Range Sum

Description: Given an integer array, count the number of range sums `sum(i,j)` that lie within `[lower, upper]` (inclusive).

Variation: Build prefix sum array. Then use modified merge sort — during the merge phase, for each element in the left half, use two sliding pointers `j` and `k` into the right half to count valid prefix sums (those where `arr[k] - arr[i]` falls in range). This gives $O(n \log n)$ vs $O(n^2)$ brute force.

Variables: `prefix` = prefix sum array (length `n+1`) · `[start, end)` half-open range over prefix · `mid` = split · `i` = left-half element (a range's start prefix) · `j / k` = sliding lower/upper bounds into the right half · `count` = valid range sums

Pseudocode:

```
build prefix sums; a range sum = prefix[hi] - prefix[lo]
mergeCount [0, n+1):
    if 0 or 1 elements: return 0
    count = mergeCount(left) + mergeCount(right)
    for each left element i:
        advance j while prefix[j]-prefix[i] < lower    // first index in range
        advance k while prefix[k]-prefix[i] <= upper  // first index past range
        count += k - j                                // right elements in [lower,upper]
    sort the two halves (merge phase) so parent sees sorted data
    return count
```

```

class Solution {
    public int countRangeSum(int[] nums, int lower, int upper) {
        int n = nums.length;
        long[] prefix = new long[n + 1];
        for (int i = 0; i < n; i++) prefix[i+1] = prefix[i] + nums[i];
        return mergeCount(prefix, 0, n + 1, lower, upper);
    }

    private int mergeCount(long[] arr, int start, int end, int lower, int upper) {
        if (end - start <= 1) return 0;
        int mid = (start + end) / 2;
        int count = mergeCount(arr, start, mid, lower, upper)
            + mergeCount(arr, mid, end, lower, upper);
        int j = mid, k = mid;
        for (int i = start; i < mid; i++) {
            while (j < end && arr[j] - arr[i] < lower) j++; // ← VARIATION: sliding lower bound
            while (k < end && arr[k] - arr[i] <= upper) k++; // ← VARIATION: sliding upper bound
            count += k - j; // count valid sums for this i
        }
        // Merge phase (sort the two halves)
        long[] sorted = Arrays.copyOfRange(arr, start, end);
        Arrays.sort(sorted);
        System.arraycopy(sorted, 0, arr, start, sorted.length);
        return count;
    }
}

```

Time $O(n \log n)$ | **Space** $O(n)$

#164 Maximum Gap

Description: Given an unsorted array, return the maximum difference between successive elements in its sorted form. Must run in linear time. **Variation:** bucket sort by pigeonhole principle. With n elements, the max gap is at least $\text{ceil}((\text{max}-\text{min})/(n-1))$. Place elements into buckets of that size; the max gap spans between one bucket's max and the next non-empty bucket's min.

Variables: min / max = value range · bucketSize = floor gap $(\text{max}-\text{min})/(n-1)$ · bucketCount = number of buckets · $\text{bucketMin}[b] / \text{bucketMax}[b]$ = extremes within bucket b · b = bucket index · previousMax = max of last non-empty bucket · maxGap = answer **Pseudocode:**

```

if fewer than 2 elements: return 0
find min and max; if equal return 0
bucketSize = max(1, (max-min)/(n-1)) // gap can't be smaller than this
create bucketCount buckets, track each bucket's min and max
for each num: drop into bucket (num-min)/bucketSize, update that bucket's min/max
previousMax = min, maxGap = 0
for each bucket b in order:
    skip empty buckets
    maxGap = max(maxGap, bucketMin[b] - previousMax) // gap straddles bucket boundary
    previousMax = bucketMax[b]
return maxGap

```



```

class Solution {
    public int maximumGap(int[] nums) {
        int n = nums.length;
        if (n < 2) return 0;
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
        for (int num : nums) { min = Math.min(min, num); max = Math.max(max, num); }
        if (min == max) return 0;
        int bucketSize = Math.max(1, (max - min) / (n - 1)); // ← VARIATION: pigeonhole bucket size
        int bucketCount = (max - min) / bucketSize + 1;
        int[] bucketMin = new int[bucketCount], bucketMax = new int[bucketCount];
        Arrays.fill(bucketMin, Integer.MAX_VALUE);
        Arrays.fill(bucketMax, Integer.MIN_VALUE);
        for (int num : nums) {
            int b = (num - min) / bucketSize;
            bucketMin[b] = Math.min(bucketMin[b], num);
            bucketMax[b] = Math.max(bucketMax[b], num);
        }
        int maxGap = 0, previousMax = min;
        for (int b = 0; b < bucketCount; b++) {
            if (bucketMin[b] == Integer.MAX_VALUE) continue; // empty bucket
            maxGap = Math.max(maxGap, bucketMin[b] - previousMax); // ← VARIATION: gap across boundary
            previousMax = bucketMax[b];
        }
        return maxGap;
    }
}

```

Time $O(n)$ | **Space** $O(n)$